🔹 **Slide 1: "The Processor: Datapath and Control - Chapter 5"**

📝 Explanation:
This introductory slide sets the stage for understanding how the processor executes instructions by combining data movement (datapath) and control logic.

💡 Key Points:
- The chapter focuses on implementing a subset of the **MIPS instruction set**.
- It explains how different types of instructions (memory, arithmetic, and branch) are handled at the hardware level.
- Introduces two major components:
  - **Datapath**: Where data flows and operations happen.
  - **Control Unit**: Directs the datapath based on the current instruction.

🎯 Teaching Tips:
- “What do you think happens inside the CPU when you run a simple program?”
- Emphasize that we’re not just learning assembly but also how the hardware executes these instructions.

🔹 **Slide 2: "A subset of the core MIPS instruction set"**

📝 Explanation:
Lists the main instruction types used in the chapter:
- Memory-reference: lw` (load word), `sw` (store word)
- Arithmetic-logical: `add`, `sub`, `and`, `or`, `slt`
- Branch: `beq (branch equal)`, `j`

These are chosen because they represent different functionalities:
- Data transfer between memory and registers.
- Computation using ALU.
- Conditional and unconditional jumps.

💡 Key Points:
- These instructions form the basis for building a complete processor.
- They differ in how they use the ALU and where they source/destinate data.

🎯 Teaching Tips:
- Show the differences between R-type, I-type, and J-type instructions briefly.

-“Which of these instructions modifies memory? Which changes the PC?”

🔹 **Slide 3–5: "An overview of the implementation"**

📝 Explanation:
Explains the general steps all instructions follow:
1. Fetch instruction from memory using the Program Counter (PC).
2. Read one or two registers based on instruction fields.
3. Use the ALU for:
   - Address calculation (`lw`/`sw`)
   - Operation execution (`add`, `sub`)
   - Comparison (`beq`)
4. Perform specific action after ALU depending on instruction type.

💡 Key Points:
- Despite differences, there’s a common structure in instruction processing.
- The ALU is central to almost every operation.

🎯 Teaching Tips:
- Draw a simple flowchart on the board showing the shared steps.
- “Why do all instructions read registers first?”

🔹 **Slide 6: "A word about Logic Convention and Clocking"**

📝 Explanation:
Introduces the concept of **clocked sequential logic**:
- Two types of elements:
  - **Combinational logic** (e.g., ALU): Output depends only on input.
  - **State elements** (e.g., Registers, Memories): Store values across clock cycles.

💡 Key Points:
- **Edge-triggered clocking** is assumed: register writes occur at the rising/falling edge.
- All combinational logic must finish within one clock cycle.

🎯 Teaching Tips:
- Use a diagram to show how data flows through combinational logic and is stored in registers.

- “What would happen if combinational logic took longer than one clock cycle?”

🔹 **Slide 7–8: "Clocking Methodology (Cont.)"**

📝 Explanation:
Further explains timing assumptions:
- One instruction takes one clock cycle.
- Execution starts on one clock edge and ends before the next.

💡 Key Points:
- Single-cycle design means the clock period must accommodate the slowest instruction.
- This leads to inefficiency but simplifies design.

🎯 Teaching Tips:
- Compare with real CPUs: “Modern processors use multi-cycle pipelines.”
- “Why might a single-cycle design be inefficient?”

🔹 **Slide 9–10: "Building a Datapath"**

📝 Explanation:
Defines what a **datapath** is:
- Collection of functional units and storage blocks that process data.
- Includes:
  - Register file
  - ALU
  - Memory
  - Multiplexers

💡 Key Points:
- Datapath supports fetching and executing instructions.
- Each component has a specific role:
  - **Register File**: Stores temporary data.
  - **ALU**: Performs computations.
  - **Memory**: Holds instructions and data.

🎯 Teaching Tips:

- Show a simplified block diagram of the datapath.
- “Where does the result of an add instruction get stored?”

🔹 **Slide 11–13: "Instruction Formats Table"**

To show how MIPS instructions are encoded in machine code using different formats. These formats define where the various components of an instruction (like registers, immediate values, and jump addresses) are located within a 32-bit word.

Format	op	rs	rt	rd	shamt	funct
R-format	6b	5b	5b	5b	5b	6b
I-format	6b	5b	5b	16b		
J-format	6b	26b				

Each column represents a bit field in the 32-bit instruction.

🔍 **Explanation of Fields**

Field Name	Meaning	Description
op	Operation Code	Always the first 6 bits; tells what type of instruction it is.
rs	Register Source	A register number used as a source operand.
rt	Register Target	Another register number, often used as a second operand or destination.
rd	Register Destination	A register number where the result will be stored (used in R-type).

shamt	Shift Amount	Used in shift instructions likesll,srl(not shown in all slides).
funct	Function Code	Further specifies the operation (e.g., add vs subtract in R-type).
immediate / address	Constant Value	In I-type, this is a 16-bit signed constant or offset.
target address	Jump Address	In J-type, this is a 26-bit address to jump to.

📐 Bit Width Summary

- Each instruction is 32 bits long.
- Total bit widths must add up to 32:

✅ R-format:

6 (op) + 5 (rs) + 5 (rt) + 5 (rd) + 5 (shamt) + 6 (funct) = 32 bits

Used for register-based operations like:

add $rd, $rs, $rt
sub $rd, $rs, $rt

✅ I-format:

6 (op) + 5 (rs) + 5 (rt) + 16 (immediate/address) = 32 bits

Used for:

- Memory access: lw, sw

Branches: beq, bne

- Example:

lw $rt, offset($rs)
beq $rs, $rt, label

✅ **J-format:**

6 (op) + 26 (target address) = 32 bits

Used for:

Unconditional jumps: j target

- Example:

j Label

💡 **Key Teaching Points**

1. Opcode Determines Instruction Type

- If op = 0 → likely R-type (needs funct to know which one).
- If op = 35 (100011) → lw
- If op = 4 (000100) → beq
- If op = 2 (000010) → j

2. Why Different Formats?

- Not all instructions need the same amount of data.
- R-type needs more register fields, but no immediate value.
- I-type uses an immediate value instead of a third register.
- J-type only needs a large address field.

🎯 **Suggested Questions for Students**

1. "How many bits are needed to represent a register number in MIPS?"
   ➤ 5 bits → supports 32 registers (numbered 0–31)
2. "Which format is used by the lw instruction? What about add?"
   ➤ lw: I-format, add: R-format

3. “Why does the jump instruction use a 26-bit address instead of full 32 bits?”
   ➤ To save space; the upper 4 bits come from the current PC.
4. “What would happen if we tried to encode an immediate value in an R-type instruction?”
   ➤ Not possible — no space for an immediate in R-format.

📝 Explanation:
Shows the layout of different MIPS instruction formats:
- **R-type** (register-based): e.g., `add $rd, $rs, $rt`
- **I-type** (immediate-based): e.g., `lw $rt, offset($rs)`
- **J-type** (jump): e.g., `j target`

💡 Key Points:
- Field positions are fixed → helps in decoding.
- Fields determine operand sources and ALU operation.

🔹 **Slide 14–16: "Datapath for R-type Instructions"**

📝 Explanation:
Focuses on how R-type instructions are implemented:
- Uses register file to read operands.
- ALU performs operation.
- Result written back to the register file.

💡 Key Points:
- ALU uses the `funct` field to decide the operation.
- ALUOp signal controls ALU behavior.

🎯 Teaching Tips:
- Walk through a sample `add $t1, $t2, $t3` instruction.
- “Which part of the instruction tells the ALU to add instead of subtract?”

🔹 **Slide 17–19: "Datapath for lw and sw Instructions"**

📝 Explanation:
For memory access:
- Calculate effective address = base register + offset.
- Load: write result to register.
- Store: write data to memory.

💡 Key Points:
- ALU used for address calculation.
- Only `lw` writes back to register; `sw` doesn’t.

🎯 Teaching Tips:
- Use color coding in the diagram to trace data flow.
- Ask: “What happens if the offset is negative?”

🔹 **Slide 20–21: "Datapath for beq Instruction"**

📝 Explanation:
Branch if equal:
- Subtract `$rs` and `$rt` using ALU.
- If zero, update PC to the branch target.
- Otherwise, PC increments normally.

💡 Key Points:
- ALU checks equality.
- Branch target calculated as: `PC + 4 + (offset << 2)`

🎯 Teaching Tips:
- Demonstrate with example: `beq $s1, $s2, Label`
- “What would happen if the ALU didn't support subtraction?”

🔹 **Slide 22–24: "Designing the Main Control Unit"**

📝 Explanation:
The control unit generates signals based on the opcode and function fields.

💡 Key Points:
- Control lines include:

Signal	Description
RegDst	Choose destination register (rt or rd)
ALUSrc	Choose second ALU operand (register or immediate)
MemtoReg	Choose write data source (memory or ALU)
RegWrite	Enable writing to register file
MemRead	Read from memory
MemWrite	Write to memory
Branch	Enable branch check
Jump	Enable unconditional jump
ALUOp	2-bit code to select ALU operation

🎯 Teaching Tips:
- Use a truth table to show control outputs for `add`, `lw`, `beq`.
- “How does the control unit know whether to write to a register?”

🔹 **Slide 25–27: "Operation of the Datapath"**

📝 Explanation:
Walkthrough of how each instruction uses the datapath and control signals.

💡 Key Points:
- Control logic decodes opcode/function to generate correct signals.
- Signals route data appropriately through multiplexers.

🎯 Teaching Tips:
- Simulate instruction execution step-by-step.
- “What would go wrong if MemWrite was asserted during a load instruction?”

🔹 **Slide 28–29: "Implementing Jumps"**

📝 Explanation:
Jump instruction (`j`) changes the PC directly.
- Uses lower 26 bits of the instruction.
- Merged with upper bits of PC+4.

💡 Key Points:
- No ALU or register operations involved.
- Simplest instruction in terms of datapath usage.

🎯 Teaching Tips:
- Explain how jump address is formed.
- “Can a jump reach any address in memory?”

🔹 **Slide 30–33: "A Multicycle Implementation"**

📝 Explanation:
Discusses limitations of single-cycle design.
- Long clock cycle due to worst-case path.
- Inefficient use of resources.

Multicycle alternative:

- Break instruction into multiple stages.
- Reuse functional units across stages.

💡 Key Points:
- Temporary registers store intermediate results between stages.
- Allows shorter clock cycles.

🎯 Teaching Tips:
- Contrast single-cycle vs multicycle.
- “What are the advantages and disadvantages of multicycle design?”

🔹 **Slide 34–36: "Exceptions"**

📝 Explanation:
Introduces exceptions — unexpected events during execution.
- Examples: undefined instructions, overflow, interrupts.

💡 Key Points:
- Exceptions require saving PC and cause register.
- Control transfers to exception handler.

🎯 Teaching Tips:
- Relate to OS concepts: system calls, traps.
- “How is an exception different from a regular branch?”

🔹 **Slide 37–39: "How Exceptions are Handled"**

📝 Explanation:
Steps to handle an exception:
1. Save address of faulting instruction in EPC.
2. Set Cause register.
3. Jump to exception handler.

💡 Key Points:
- Exception handling is done in software.
- Hardware provides necessary information.

🔹 **Slide 40–41: "Exceptions"**

📝 **Explanation:**

Introduces the concept of exceptions , which are unexpected events that occur during instruction execution.

- Types of exceptions:
  - Undefined instructions
  - Arithmetic overflow
  - External interrupts (from hardware)

💡 **Key Points:**

- Exceptions interrupt normal program flow.
- They are handled by exception handlers in software (OS or firmware).
- The processor must save enough state to resume execution after handling the exception.

🎯 **Teaching Tips:**

- Relate to real-world examples: division by zero, invalid memory access.
- “How is an exception different from a function call?”

🔹 **Slide 42–44: "How Exceptions are Handled"**

📝 **Explanation:**

Describes the hardware support for handling exceptions:

1. Save the address of the faulting instruction in EPC (Exception Program Counter).
2. Set the Cause register to indicate the type of exception.
3. Jump to a predefined exception handler address (typically `0x80000080`).

💡 **Key Points:**

- Exception handling is a combination of hardware and software.
- Hardware saves context; software handles the actual error or event.
- After handling, the program can either terminate or resume.

🎯 **Teaching Tips:**

- Use a simple example like divide-by-zero to demonstrate the flow.
- “What would happen if the EPC wasn’t saved properly?”

🔹 **Slide 45–47: "Implementing Jumps"**

📝 **Explanation:**

Explains how jump (j) instructions work:

- Opcode = 2 for jump.
- The target address is formed using:
  - Upper 4 bits of current PC + 4
  - The lower 26 bits from the instruction field
- Final address = (PC + 4)[31:28] || target_address || 00

Explanation:

This section explains how the unconditional jump instruction (j) is implemented at the hardware level in a MIPS processor. Unlike other instructions like add, lw, or beq, the j instruction changes the Program Counter (PC) directly to a new address, allowing the program to "jump" to a different part of memory.

📝 **Key Explanation**

✅ **Instruction Format:**

The J instruction uses J-format, which looks like this:

op (6 bits)	target address (26 bits)
000010 (for j)	26-bit address

- Opcode = 2 (binary: 000010) → indicates it's a jump.
- The remaining 26 bits specify the target address .

⚙️ How the Target Address is Calculated:

Jump addresses in MIPS are not 32-bit full addresses. Instead, they're 26-bit values embedded in the instruction. To compute the final 32-bit address, the following steps are taken:

Step-by-step:

1. Take PC + 4
    - The current value of the PC is always pointing to the next instruction, so we use PC + 4 as the base.
    - This ensures correct behavior even during pipelining.
2. Extract the upper 4 bits of (PC + 4):
    - These bits come from positions [31:28] of the updated PC.
3. Combine with the 26-bit target field from the instruction:
    - From the instruction, take the 26-bit target address field.
4. Shift the 26-bit address left by 2 bits (×4):
    - Since all MIPS instructions are aligned on 4-byte boundaries, the two least significant bits are always 00.
5. Final Jump Address Calculation:

    Final Address = (PC + 4)[31:28] || (Instruction[25:0]) << 2

6. Or visually:

    [Upper 4 bits of PC+4][26-bit target address from instruction][00]

🎯 Example:

Let's say:

- Current PC = 0x00400000
- Instruction is: j Label where the target field is 0x000000 (i.e., all zeros)

Then:

1. PC + 4 = 0x00400004
2. Upper 4 bits of PC+4 = 0000
3. Target from instruction = 0x000000 → binary: 000000...000000 (26 bits)
4. Shift left by 2: becomes 000000...00000000 (28 bits)
5. Final address = 0000 00000000000000000000000000 00 = 0x00000000

So, the CPU will jump to address 0x00000000.

💡 **Key Points:**

- No ALU or register file is involved in the jump.
- Simplest instruction in terms of datapath usage.
- Limited to 256MB address space due to 26-bit encoding.

🎯 **Teaching Tips:**

- Walk through a sample jump instruction calculation.
- Ask: “Why can't jump reach all possible addresses in memory?”

🔹 **Slide 48–50: "Why a Single-cycle Implementation is Not Used?"**

📝 **Explanation:**

Discusses limitations of the single-cycle processor design:

- The clock cycle must be long enough to accommodate the slowest instruction.
- Leads to inefficient use of time and power.
- All components must complete within one clock cycle → limits performance.

💡 **Key Points:**

- Single-cycle processors are simple but not scalable.
- Real CPUs use pipelining or multi-cycle designs.

🎯 **Teaching Tips:**

- Compare with multi-cycle design (next section).
- “If all instructions take the same amount of time, why is this inefficient?”

🔹 **Slide 51–55: "A Multicycle Implementation"**

📝 **Explanation:**

Introduces multicycle implementation where:

- One instruction spans multiple clock cycles.
- Functional units are reused across stages.

💡 **Key Points:**

- Breaks instruction into steps:
    1. Instruction Fetch
    2. Instruction Decode / Register Fetch
    3. Execute / Address Calculation
    4. Memory Access
    5. Write Back
- Temporary registers store intermediate results between stages.

🎯 **Teaching Tips:**

- Use a table to show how each instruction uses different stages.
- “What’s the advantage of reusing the ALU across multiple stages?”

🔹 **Slide 56–58: "Why the Temporary Registers Are Used?"**

📝 **Explanation:**

Temporary registers (like IR, A, B, ALUOut, etc.) are used to:

- Store values between clock cycles.
- Allow reuse of functional units across stages.

💡 **Key Points:**

- Needed because data can't pass through multiple stages in one clock cycle.
- Enables pipelining-like behavior without full pipelining.

🎯 **Teaching Tips:**

- Show how temporary registers help in storing operands and results.
- "What would happen if we didn't have these registers?"

🔹 **Slide 59–61: "Comparison with Single-cycle Implementation"**

📝 **Explanation:**

Compares multicycle vs single-cycle:

Feature	Single-cycle	Multicycle
Clock cycle	Long (slowest instruction)	Shorter (one stage per cycle)
Performance	Low (wasted time)	Higher (efficient use of time)
Complexity	Simple	More complex control logic
Throughput	One instruction per cycle	One instruction every few cycles

💡 **Key Points:**

- Multicycle allows better utilization of hardware.
- Control logic becomes more complex but performance improves.

🎯 **Teaching Tips:**

- Discuss trade-offs in design choices.
- “Would you prefer simplicity or performance in a CPU design?”

🔹 **Slide 62–64: "With the Additional Multiplexers"**

📝 **Explanation:**

Shows how multiplexers are added to route data correctly between stages and components.

- Multiplexers choose:
  - Between the register file output and the ALU result
  - Between the memory output and the ALU result
  - Between PC+4 and branch/jump target

💡 **Key Points:**

- Multiplexers enable flexibility in data routing.
- Controlled by signals from the control unit.

🎯 **Teaching Tips:**

- Use color coding on the board to trace data paths.
- “Which signal decides whether the ALU result goes to memory or a register?”

🔹 **Slide 65–67: "With the Control Lines Added"**

📝 **Explanation:**

Introduces control lines that drive:

- Multiplexers
- Register write enable
- Memory read/write
- ALU operation

- Branch decision

💡 **Key Points:**

- Control logic decodes opcode and funct fields.
- Each instruction type has a unique set of control signals.

🎯 **Teaching Tips:**

- Show how control signals change for add, lw, beq.
- “How many total control signals do we need in this design?”